

Example 1:

The findMatches method in my NextDestination.java file was originally written with a single loop that combined multiple responsibilities such as matching destinations by weather and budget and checking if all the desired activities were present.

While functional, this structure made the code harder to read, maintain, and extend. To address these issues, I decided to refactor the method by breaking it down into smaller, reusable methods. The goal was to simplify the logic, improve readability, and make future updates easier.

Changes Made:

1. Extracted the code responsible for comparing a destination's weather and budget into a new helper method called matchesWeatherAndBudget.
2. Moved the logic for checking if all required activities are present in a destination into another helper method called matchesActivities.
3. Updated the findMatches method to use these helper methods, making the main loop shorter and more descriptive.

Before Refactoring:

```
private List<Destination> findMatches(String weather, String budget, List<String> activities) {  
    List<Destination> matches = new ArrayList<>();  
  
    for (Destination dest : destinations) {  
        if (dest.getWeather().equals(weather) && dest.getBudget().equals(budget)) {  
            boolean allActivitiesPresent = true;  
            for (String activity : activities) {  
                if (!dest.getActivities().contains(activity)) {  
                    allActivitiesPresent = false;  
                    break;  
                }  
            }  
  
            if (allActivitiesPresent) {  
                matches.add(dest);  
            }  
        }  
    }  
  
    return matches;  
}
```

While this worked, the code was difficult to read and combined too many tasks in one place.

After Refactoring:

```
private List<Destination> findMatches(String weather, String budget, List<String> activities) {
    List<Destination> matches = new ArrayList<>();

    for (Destination dest : destinations) {
        if (matchesWeatherAndBudget(dest, weather, budget) && matchesActivities(dest, activities)) {
            matches.add(dest);
        }
    }

    return matches;
}

private boolean matchesWeatherAndBudget(Destination dest, String weather, String budget) {
    return dest.getWeather().equals(weather) && dest.getBudget().equals(budget);
}

private boolean matchesActivities(Destination dest, List<String> activities) {
    for (String activity : activities) {
        if (!dest.getActivities().contains(activity)) {
            return false;
        }
    }
    return true;
}
```

Why I made these changes:

- The new code is easier to read because each method has a clear, descriptive name.
- The logic in findMatches is no longer cluttered with nested loops and conditions.
- matchesWeatherAndBudget and matchesActivities can now be reused elsewhere if needed.
- By separating the logic into smaller methods, I can write unit tests for each helper method individually.
- If the filtering criteria change, I can simply create another helper method without modifying the existing ones.

Example 2:

In my NextDestination.java file, the activity options were hard-coded directly into the start method. While this worked, it wasn't an ideal solution. Hard-coding these options in the method made the code harder to maintain because any updates would require editing the method directly. It also meant the activity options couldn't easily be reused elsewhere in the project.

To improve this, I refactored the code by moving the activity options into a **constant field** at the class level. This centralizes the activity options, making the code easier to manage and more consistent.

Changes Made:

1. Moved the list of activity options out of the start method and into a constant named `ACTIVITY_OPTIONS` at the top of the `NextDestination` class.
2. Replaced the hard-coded array inside the start method with a reference to the new constant.

Before Refactoring:

```
public void start() {  
    String[] weatherOptions = {"cold", "warm", "hot"};  
    String[] budgetOptions = {"low", "medium", "high", "unlimited"};  
    String[] activityOptions = {"beach", "hiking", "sightseeing", "culture", "food", "shopping", "museums", "northern lights", "temples"};
```

These options were directly defined in the method, making it harder to update or reuse elsewhere.

```
List<String> activities = getActivities(activityOptions);
```

After Refactoring:

```
public class NextDestination {  
  
    private static final String[] activityOptions = {"beach", "hiking", "sightseeing", "culture", "food", "shopping", "museums", "northern lights", "temples"};  
    private List<Destination> destinations;  
  
    List<String> activities = getActivities(activityOptions);
```

Why I made these changes:

The main reason for this refactoring was to make the code more maintainable and consistent. By moving the activity options into a constant, I now only need to update the options in one place. This ensures that the options are always the same, no matter where they're used in the project.

It also makes the start method cleaner, as it no longer includes the detailed array definition. Instead, it focuses on the logic of the program.